

Trabajo Práctico Integrador

Sistema de Gestión de Delivery

MORFI

Programación I — Ingeniería de Software I — Base de Datos I

Integrantes: Damian Martinez - Nahuel Gonzalez - Elian Rosito

Sosa

Universidad Atlántida Argentina

Índice

1. Introducción.....	3
2. Diseño del Sistema.....	4
2.1 Estructura de módulos.....	4
2.2 Modelo de datos.....	5
3. Funciones Clave.....	6
3.1 Inicialización de la base de datos.....	6
3.2 Lectura y escritura de archivos binarios.....	6
3.3 Búsqueda por ID.....	7
3.4 Eliminación con archivo auxiliar.....	7
3.5 Asignación automática de IDs.....	7
3.6 Sistema de múltiples roles.....	8
4. Listado de Funciones.....	9
4.1. crud.c — Gestión de usuarios.....	9
4.2. crud.c — Gestión de roles.....	9
4.3. crud.c — Restaurantes, productos y pedidos.....	10
4.4. BBD.c — Paneles de usuario.....	11
4.5. filemanager.c — Manejo de archivos.....	11
4.6. ui.c — Interfaz visual.....	11
5. Informes en Texto.....	12
6. Validaciones.....	13
7. Dificultades Encontradas.....	14
División de tareas.....	14
Coordinación de horarios.....	14
Diferencias en la experiencia técnica.....	14
Gestión de múltiples roles por usuario.....	14
Modificación de registros en archivos binarios.....	15
Restricciones de tiempo.....	15
Complejidad de la integración.....	15
8. Conclusiones.....	16

1. Introducción

En la actualidad, el crecimiento de los servicios de entrega a domicilio ha generado la necesidad de contar con herramientas que permitan gestionar de manera eficiente los pedidos, usuarios y recursos involucrados en este proceso. En este contexto, se desarrolló MORFI, un sistema de gestión de delivery implementado en lenguaje C, con el objetivo de centralizar y optimizar las operaciones relacionadas con la administración de pedidos y usuarios.

La plataforma contempla cuatro tipos de usuarios: Clientes, Repartidores, Encargados de Restaurante y Administradores, cada uno con funcionalidades y niveles de acceso específicos. Esta estructura permite organizar las responsabilidades de cada actor y garantizar un control adecuado de la información.

El sistema implementa una base de datos basada en el Modelo Entidad-Relación desarrollado durante la cursada de Base de Datos I, que consta de 21 entidades identificadas en la materia. Toda la información se persiste en archivos binarios de registros de longitud fija, simulando el comportamiento de las tablas del modelo relacional. La arquitectura, los requerimientos y el diseño general del sistema fueron definidos siguiendo los conceptos y metodologías estudiados en Ingeniería de Software I.

La funcionalidad principal de MORFI se centra en la gestión integral de pedidos, desde su creación hasta su entrega final. Además, el sistema permite administrar restaurantes y productos, gestionar pagos y comisiones, generar informes en archivos de texto, y registrar calificaciones de los clientes.

2. Diseño del Sistema

El sistema MORFI fue desarrollado siguiendo un enfoque modular, separando las distintas funcionalidades en archivos fuente (.c) y archivos de cabecera (.h). Esta organización facilita el mantenimiento, la reutilización del código y la escalabilidad del proyecto.

2.1 Estructura de módulos

Módulo	Archivos	Descripción
Principal	main.c	Menú de selección de aplicación y flujo principal del programa
Base de Datos	BBD.c / BBD.h	Definición de todas las estructuras (typedef struct) y paneles de usuario por rol
CRUD	crud.c / crud.h	Altas, bajas, modificaciones y consultas sobre todos los archivos binarios
Manejador de archivos	filemanager.c / filemanager.h	Funciones genéricas para abrir, leer, escribir y buscar en archivos
Informes	informes.c / informes.h	Generación de los cuatro reportes en archivos de texto
Validaciones	validaciones.c / validaciones.h	Validación de email, contraseña, teléfono, nombre y horarios
Interfaz	ui.c / ui.h	Funciones de presentación visual: cajas, menús

2.2 Modelo de datos

Las estructuras de datos fueron definidas en BBD.h respetando el modelo relacional obtenido en Base de Datos I. Cada entidad se almacena en su propio archivo binario de registros de longitud fija:

Entidad	Archivo .dat	Campos principales
Usuario	usuario.dat	id_usuario, nombre, apellido, email, contrasena
Cliente	cliente.dat	id_usuario, direccion
Repartidor	repartidor.dat	id_usuario, vehiculo, estado_disponibilidad
Encargado	encargado.dat	id_usuario, id_restaurante, turno
Administrador	administrador.dat	id_usuario, nivel_acceso
Restaurante	restaurante.dat	id_restaurante, nombre, horario, estado
UbicacionRestaurante	ubicacion_restaurante.dat	id_ubicacion, id_restaurante, direccion, ciudad
Producto	producto.dat	id_producto, id_menu, nombre, precio, disponibilidad
Pedido	pedido.dat	id_pedido, id_cliente, id_restaurante, estado, total
OrdenPreparacion	orden_preparacion.dat	id_orden, id_pedido, estado_preparacion
DetalleOrden	detalle_orden.dat	id_orden, id_producto, cantidad
HistorialEstado	historial_estado.dat	id_historial, id_pedido, estado, fecha
Calificacion	calificacion.dat	id_calificacion, puntuacion_comida, puntuacion_repartidor
Pago	pago.dat	id_pago, id_pedido, monto_total, metodo_pago
ComisionDetalle	comision.dat	id_comision, monto_pedido, porcentaje, monto_netto

Al iniciar el sistema, la función abrirBDD() crea automáticamente todos los archivos .dat en modo "ab" si no existen, garantizando que el sistema arranque correctamente en cualquier entorno sin configuración previa (Los faltantes no fueron puestos en el sistema).

3. Funciones Clave

A continuación se describen las funciones más representativas del sistema, con fragmentos de código que ilustran los patrones de implementación utilizados.

3.1 Inicialización de la base de datos

Al iniciar el programa se invoca `abrirBDD()`, que crea todos los archivos `.dat` en modo "ab" si no existen. Esto garantiza que el sistema arranque correctamente sin necesidad de configuración previa.

```
// crud.c - abrirBDD()
void abrirBDD() {
    FILE *archivo;
    archivo = fopen("usuario.dat", "ab");
    if (archivo != NULL) fclose(archivo);
    archivo = fopen("restaurante.dat", "ab");
    if (archivo != NULL) fclose(archivo);
    // ... (repite para cada tabla del sistema)
}
```

3.2 Lectura y escritura de archivos binarios

Todas las operaciones de persistencia utilizan `fwrite` y `fread` con estructuras de longitud fija. Para agregar un cliente nuevo se escribe simultáneamente en `usuario.dat` y `cliente.dat`, verificando el resultado de cada escritura:

```
// crud.c - agregarCliente()
int resultado_usuario = fwrite(&usuario, sizeof(Usuario),
1, archivoUsuario);
int resultado_cliente = fwrite(&cliente, sizeof(Cliente),
1, archivoCliente);
fflush(archivoUsuario);
fflush(archivoCliente);
```

Para la modificación in-place de registros se utiliza `fseek` para posicionar el cursor sobre el registro encontrado y sobrescribirlo, sin necesidad de reescribir todo el archivo:

```
// crud.c - actualizarClienteXid()
fseek(archivo, -(long)sizeof(Cliente), SEEK_CUR);
fwrite(&cliente, sizeof(Cliente), 1, archivo);
```

3.3 Búsqueda por ID

La búsqueda de registros se realiza recorriendo el archivo secuencialmente con `fread` hasta encontrar el registro que coincide con el criterio buscado, sin cargar todo el contenido en memoria. Este patrón se aplica de forma consistente en todo el sistema:

```
// Patrón de búsqueda por ID - aplicado en crud.c
while (fread(&usuario, sizeof(Usuario), 1, archivo) == 1) {
    if (usuario.id_usuario == id_buscado) {
        // registro encontrado
        fclose(archivo);
        return 1;
    }
}
```

3.4 Eliminación con archivo auxiliar

Para la baja de registros se emplea el patrón de archivo auxiliar: se copian al archivo temporal todos los registros excepto el que se desea eliminar, luego se reemplaza el original. Este método se aplica en las bajas de usuarios, restaurantes, productos y roles:

```
// crud.c - eliminarCliente()
while (fread(&cliente, sizeof(Cliente), 1, archivo) == 1) {
    if (cliente.id_usuario != id_usuario)
        fwrite(&cliente, sizeof(Cliente), 1, auxiliar);
}
fclose(archivo);
fclose(auxiliar);
remove("cliente.dat");
rename("auxiliar_cliente.dat", "cliente.dat");
```

3.5 Asignación automática de IDs

Los IDs se generan buscando el valor máximo existente en el archivo y sumando 1. Esto evita duplicados sin necesitar un contador externo o un archivo de configuración adicional:

```
// crud.c - obtenerSiguietePedidoId()
int obtenerSiguietePedidoId() {
    FILE *archivo = fopen("pedido.dat", "rb");
    Pedido pedido; int maxId = 0;
    if (archivo != NULL) {
        while (fread(&pedido, sizeof(Pedido), 1, archivo) == 1) if
            (pedido.id_pedido > maxId) maxId = pedido.id_pedido;
        fclose(archivo);
    }
    return maxId + 1;
}
```

3.6 Sistema de múltiples roles

Una característica destacada del diseño es que un mismo usuario puede tener múltiples roles simultáneamente. Al iniciar sesión, el sistema detecta cuántos roles tiene el usuario y, si tiene más de uno, le permite elegir con cuál ingresar. Esto se implementa mediante funciones de verificación de rol que consultan cada archivo binario:

```
// crud.c - obtenerRolesDelUsuario()
void obtenerRolesDelUsuario(int id_usuario, int roles[4]) {
    roles[0] = esCliente(id_usuario);
    roles[1] = esEncargado(id_usuario);
    roles[2] = esRepartidor(id_usuario);
    roles[3] = esAdministrador(id_usuario);
}
```


4. Listado de Funciones

4.1. crud.c — Gestión de usuarios

Función	Descripción
<code>abrirBDD()</code>	Inicializa los archivos .dat del sistema
<code>agregarUsuario(usuario, opcion_app)</code>	Registra un nuevo usuario con el rol correspondiente a la app seleccionada
<code>actualizarUsuarioXid(usuario)</code>	Modifica los datos de un usuario existente
<code>eliminarUsuario(usuario)</code>	Elimina un usuario y todos sus roles asociados
<code>eliminarUsuarioPorId(usuario_actual)</code>	Permite al administrador eliminar una cuenta por ID con confirmación
<code>listarUsuarios()</code>	Muestra el listado completo de usuarios registrados
<code>seleccionarCuenta(usuario)</code>	Verifica credenciales y redirige al panel correspondiente

4.2. crud.c — Gestión de roles

Función	Descripción
<code>agregarCliente / Repartidor / Encargado / Administrador()</code>	Asigna el rol específico al usuario
<code>actualizarClienteXid / RepartidorXid / EncargadoXid()</code>	Actualiza datos del rol indicado
<code>eliminarCliente / Repartidor / Encargado / Administrador()</code>	Remueve la asociación de rol
<code>esCliente / esEncargado / esRepartidor / esAdministrador()</code>	Verifica si el usuario tiene el rol indicado
<code>obtenerIdPorEmail(email)</code>	Obtiene el ID de un usuario por su email
<code>contarRolesDelUsuario(id_usuario)</code>	Retorna la cantidad de roles que tiene un usuario
<code>obtenerRolesDelUsuario(id_usuario, roles[])</code>	Llena un arreglo con los roles activos del usuario
<code>agregarRolExistente(id_usuario)</code>	Agrega un nuevo rol a una cuenta ya registrada

4.3. crud.c — Restaurantes, productos y pedidos

Función	Descripción
<code>agregarRestaurante()</code>	Registra un nuevo restaurante con su ubicación
<code>eliminarRestaurantePorId()</code>	Elimina un restaurante con confirmación
<code>listarRestaurantes()</code>	Muestra todos los restaurantes con sus ubicaciones
<code>agregarProducto(id_restaurante)</code>	Agrega un producto al menú del restaurante
<code>modificarProducto(id_restaurante)</code>	Modifica un producto existente
<code>listarProductosRestaurante(id_restaurante)</code>	Lista los productos disponibles de un restaurante
<code>agregarPedido(usuario)</code>	Registra un nuevo pedido con sus detalles y pago
<code>listarMisPedidos(usuario)</code>	Muestra los pedidos del cliente
<code>listarPedidoDisponibles(usuario)</code>	Muestra pedidos disponibles para repartidores
<code>aceptarPedido(usuario)</code>	Asigna un repartidor a un pedido
<code>actualizarEstadoOrden(id_restaurante)</code>	Actualiza el estado de preparación de una orden
<code>obtenerHistorialEstados(id_pedido)</code>	Muestra el historial de estados de un pedido
<code>realizarPago(usuario, id_pedido)</code>	Procesa y registra el pago de un pedido
<code>calificarPedido(usuario)</code>	Registra una calificación para un pedido entregado

4.4. BBD.c — Paneles de usuario

Función	Descripción
<code>panelCliente(usuario)</code>	Interfaz del cliente: pedidos, pagos, historial y calificaciones
<code>panelRepartidor(usuario)</code>	Interfaz del repartidor: entregas disponibles y pedidos aceptados
<code>panelEncargador(usuario)</code>	Interfaz del encargado: menú del restaurante y órdenes
<code>panelAdministrador(usuario)</code>	Interfaz del administrador: usuarios, restaurantes y reportes

4.5. filemanager.c — Manejo de archivos

Función	Descripción
<code>abrirArchivo(nombre)</code>	Abre un archivo y verifica su disponibilidad
<code>leerRegistro(nombre)</code>	Recupera y muestra un registro de un archivo
<code>escribirArchivo(nombre, texto)</code>	Almacena información en un archivo
<code>busquedaXid(nombre, id)</code>	Busca un registro por ID en un archivo
<code>listarArchivo(nombre)</code>	Muestra el contenido completo de un archivo

4.6. ui.c — Interfaz visual

Función	Descripción
<code>inicializarUI()</code>	Configura el entorno visual al inicio del programa
<code>limpiarPantalla()</code>	Limpia la consola
<code>mostrarCaja(titulo)</code>	Dibuja un marco con título para cada sección
<code>mostrarOpcion(numero, texto)</code>	Muestra una opción de menú numerada
<code>mostrarError(mensaje)</code>	Presenta mensajes de error
<code>mostrarExito(mensaje)</code>	Confirma operaciones exitosas
<code>mostrarInput(prompt)</code>	Solicita el ingreso de datos al usuario
<code>pausar()</code>	Detiene la ejecución hasta que el usuario continúe

5. Informes en Texto

Se implementaron los cuatro informes requeridos por el trabajo práctico. Todos se generan con `fprintf` sobre archivos `.txt` e incluyen encabezados y separadores para facilitar su lectura. La fecha del informe se obtiene utilizando la biblioteca `time.h`.

Función	Archivo generado	Descripción
<code>listadoCalificacion()</code>	<code>restaurantes_top.txt</code>	Lista los restaurantes con calificación promedio mayor a 4.0, calculada a partir de las puntuaciones de comida, repartidor y entrega
<code>pedidosUsuario(id)</code>	<code>reporte_pedidos_usuario.txt</code>	Historial completo de pedidos de un usuario: ID, fecha, estado y dirección de entrega
<code>platosMasPedidos()</code>	<code>top_platos.txt</code>	Los 5 productos con mayor cantidad de pedidos, obtenidos sumando las cantidades del archivo <code>detalle_orden.dat</code>
<code>facturaPedido(id)</code>	<code>factura_pedido.txt</code>	Detalle completo de un pedido: productos, cantidades y monto total

Ejemplo de estructura de salida del informe `top_platos.txt`:

```
// top_platos.txt
===== TOP 5 PLATOS MAS PEDIDOS =====

1) Milanesa napolitana - 12 unidades
2) Empanadas (x6) - 9 unidades
3) Pizza muzarella - 7 unidades
...
```

6. Validaciones

El módulo validaciones.c centraliza todas las verificaciones de datos de entrada, garantizando la integridad de la información antes de ser persistida en los archivos binarios.

Función	Criterio validado
<code>validarEmail(email)</code>	Debe contener @ y al menos un punto posterior
<code>validarContrasena(contrasena)</code>	Mínimo 6 caracteres
<code>validarTelefono(telefono)</code>	Solo caracteres numéricos, longitud mínima de 7 dígitos
<code>validarNombre(nombre)</code>	No acepta cadenas vacías ni que comiencen con espacios
<code>validarHorario(horario)</code>	Formato HH:MM con horas entre 0-23 y minutos entre 0-59
<code>validarEstadoRestaurante(estado)</code>	Solo acepta: activo, inactivo, mantenimiento
<code>esTextoValido(texto)</code>	Verifica que la cadena contenga solo caracteres alfabéticos permitidos
<code>esNumeroValido(texto)</code>	Verifica que la cadena represente un valor numérico válido
<code>convertirEspaciosAGuiones(texto)</code>	Normaliza cadenas reemplazando espacios por guiones para el almacenamiento

Además de las validaciones de formato, se implementaron validaciones de lógica de negocio: no se permite registrar un usuario con un email ya existente, no se puede agregar un rol que el usuario ya posee, no se permite al administrador eliminar su propio usuario, y al calificar se verifica que el pedido pertenezca al usuario autenticado.

7. Dificultades Encontradas

7.1. División de tareas

Durante el desarrollo del proyecto, la asignación de tareas se realizó principalmente en función de las necesidades que surgían día a día. Si bien este enfoque permitió responder rápidamente a los requerimientos inmediatos, también generó cierta desorganización en la planificación del trabajo. Como consecuencia, en algunas ocasiones se produjeron superposiciones de tareas o correcciones repetidas sobre los mismos componentes del sistema.

7.2. Coordinación de horarios

Uno de los desafíos más importantes fue la coordinación entre los integrantes del equipo, ya que cada miembro disponía de horarios diferentes para trabajar en el proyecto. Para mitigar este problema, se adoptó una dinámica de trabajo escalonada donde los avances se realizaban en distintos momentos del día, permitiendo mantener un progreso continuo en el desarrollo.

7.3. Diferencias en la experiencia técnica

Los integrantes del grupo contaban con distintos niveles de conocimiento en el lenguaje C. Esta situación generó dificultades tanto en la implementación de determinadas funcionalidades como en la comprensión del código desarrollado. Sin embargo, también representó una oportunidad de aprendizaje colaborativo, favoreciendo el intercambio de conocimientos entre los miembros del equipo.

7.4. Gestión de múltiples roles por usuario

El mayor desafío de diseño fue implementar el sistema de roles múltiples. Inicialmente se pensó en un campo "rol" dentro de la estructura Usuario, pero esto no permitía que un mismo usuario tuviera más de un rol simultáneamente. La solución fue separar cada rol en su propio archivo binario y verificar la existencia del ID en cada uno, lo que permitió una gestión de roles completamente flexible.

7.5. Modificación de registros en archivos binarios

La modificación in-place de registros con fseek resultó más compleja de lo esperado. Se tuvo que trabajar con la secuencia correcta de modos de apertura y el uso de fseek con SEEK_CUR para retroceder exactamente el tamaño de la estructura antes de sobrescribir, evitando corromper registros adyacentes.

7.6. Restricciones de tiempo

El tiempo disponible para completar el proyecto resultó limitado en relación con el alcance de las funcionalidades propuestas. Gran parte del esfuerzo se destinó a la detección y corrección de errores, lo que redujo el tiempo disponible para incorporar nuevas funcionalidades planificadas inicialmente.

7.7. Complejidad de la integración

A medida que el proyecto fue creciendo, la integración entre los distintos módulos presentó nuevos desafíos. La necesidad de garantizar la correcta interacción entre usuarios, roles, pedidos, restaurantes, pagos e informes requirió numerosas pruebas y correcciones para asegurar la estabilidad del sistema.

8. Conclusiones

El desarrollo de MORFI permitió integrar de forma práctica los contenidos de tres materias simultáneas. La experiencia de trabajar con archivos binarios en C hizo evidente la importancia del diseño previo de las estructuras de datos: un campo mal dimensionado o un tipo de dato incorrecto en la estructura puede corromper todos los registros almacenados.

La modularización del código demostró ser clave para poder dividir el trabajo en equipo y mantener la coherencia del proyecto a lo largo de las distintas versiones. El uso de archivos .h como contratos entre módulos facilitó enormemente la integración de las partes desarrolladas por cada integrante.

El sistema de roles múltiples fue la decisión de diseño más compleja del proyecto y también la más enriquecedora. Pensar en un usuario que puede ser cliente y encargado al mismo tiempo obligó a replantear la estructura de datos desde cero y llevó a una solución más robusta y flexible que la planteada inicialmente.

Como aprendizaje principal, la normalización trabajada en Base de Datos I tuvo un impacto directo y concreto en la implementación: cada tabla normalizada se convirtió en una estructura C y un archivo .dat, haciendo evidente la relación entre el diseño teórico del modelo relacional y su implementación práctica en código. Este proyecto demostró que las tres materias no son independientes sino que se complementan y dependen entre sí.